

TrioSim: 面向多 GPU 系统大规模 DNN 负载的轻量级模拟器

TrioSim: A Lightweight Simulator for Large-Scale DNN Workloads on Multi-GPU Systems

Ying Li
William & Mary
yli81@wm.edu

Yuhui Bao*
Northeastern
University
bao.yu@northeastern.edu

Gongyu Wang
Lightmatter
wgongyu@gmail.com

Xinxin Mei
Jefferson Lab
xmei@jlab.org

Pranav Vaid*
Stanford University
pvaid@cs.stanford.edu

Anandaroop Ghosh
Lightmatter
anandaroopg08@gmail.com

Adwait Jog
University of
Virginia
ajog@virginia.edu

Darius Bunandar
Lightmatter
darius@lightmatter.co

Ajay Joshi
Lightmatter / Boston
University
ajay@lightmatter.co

Yifan Sun
William & Mary
ysun25@wm.edu

* 本工作的部分内容是 Yuhui Bao 与 Pranav Vaid 在 Lightmatter 实习期间完成的。本作品采用 Creative Commons Attribution-NoDerivatives 4.0 International 许可协议授权。

摘要

深度学习（Deep Neural Network, DNN）在从图像识别到内容生成等任务上的能力日益强大。DNN 的训练与推理高度依赖 GPU，因为 GPU 大规模并行的体系结构提供了极高的计算能力。随着 DNN 复杂度与训练数据集规模的增长，用大量 GPU 训练 DNN 正成为主流策略。研究人员一直在探索如何为 GPU 机群设计软硬件系统，以在训练或推理中获得最佳利用率、效率与 DNN 精度。然而，在设计部署此类系统时，设计者通常依赖在配备大量 GPU 的物理硬件平台上做测试，成本高昂，即使对资源极为充裕的公司而言，测试不同配置与设计的代价也几乎难以承受。虽然替代方案是在 GPU 模拟器上测试，但它们对这类大规模系统往往过于缓慢，且依赖从真实分布式系统采集的画像细节来启动模拟。为应对这些挑战，我们提出 TrioSim——一个面向多 GPU 系统 DNN 的新型轻量级模拟器。TrioSim 结合性能建模技术与模拟方法，实现了高灵活性、高模拟速度与高精度。TrioSim 仅依赖在单块 GPU 上采集的算子级执行轨迹，从而把用户所需输入降到最少，并能模拟由复杂、非对称互连连接多块 GPU 的新软硬件设计。TrioSim 可在数秒内完成模拟，预测多 GPU 系统上 DNN 训练的执行时间，数据并行、张量并行与流水线并行的平均误差分别为 2.91%、4.54% 和 6.82%。

CCS 概念: • 计算方法 → 模拟工具。

关键词: 深度学习; 多 GPU 系统; 模拟

ACM 引用格式: Ying Li, Yuhui Bao, Gongyu Wang, Xinxin Mei, Pranav Vaid, Anandaroop Ghosh, Adwait Jog, Darius Bunandar, Ajay Joshi, and Yifan Sun. 2025. TrioSim: A Lightweight Simulator for Large-Scale DNN Workloads on Multi-GPU Systems. In Proceedings of the 52nd Annual International Symposium on Computer Architecture (ISCA '25), June 21–25, 2025, Tokyo, Japan. ACM, New York, NY, USA, 15 pages. <https://doi.org/10.1145/3695053.3731082>

1 引言

如今，深度学习（DNN）是高性能计算平台上最常见的一类负载，这些平台配备 GPU [6, 18, 22, 27]。凭借大规模并行的体系结构，GPU 能为 DNN 训练与推理提供极高的计算能力 [47, 53]。由于训练数据集与模型规模庞大 [11, 46, 69]，多 GPU 系统如今被普遍用于 DNN 训练与推理。然而，多 GPU 系统也带来了数据管理、通信与同步方面的新挑战。GPU 之间慢速网络造成的开销，可能抵消增加 GPU 带来的性能提升。

多年来，人们开发了多种新颖的硬件技术 [10, 37, 42, 54, 61, 76]，以应对在多 GPU 系统上高效运行 DNN 负载的挑战。然而，研究人员往往依赖真实 GPU 硬件测试床验证设计技术，成本高昂。在许多情况下，若研究设想涉及硬件修改，忠实验证该设想几乎不可能。这些缺点促使人们开发低成本、低门槛且具有合理保真度的方法，来评估机器学习系统、内存管理方案与调度机制。

一种可能的方案是用 GPU 模拟器（如 Accel-Sim [28]、MGPUSim [4, 67]、Multi2Sim [72]、gem5 AMD GPU 模型 [62]）评估 DNN 系统。然而，这些周期级模拟器对大规模 DNN 负载与多 GPU 平台而言太慢。正如 Principal Kernel Analysis [1] 一文所估计，在 GPU 上模拟此类负载可能需要数百年，使其不可用。近期研究 [1, 40, 45] 也在开发基于采样的方法来跳过部分模拟，使基于周期的模拟器得以模拟 DNN 负载。然而，在识别执行阶段与采样时机时，它们仍相当耗费资源。

数据驱动性能模型（如 AstraSim [60, 74]、DistSim [41]、vTrain [3]）的近期进展快速且精度合理。然而，现有方案的一个主要问题是：它们严重依赖包含 GPU-GPU 通信任务的画像轨迹。这一要求带来两个问题：1) 用户必须拥有多 GPU 平台才能采集轨迹，抬高了使用门槛；2) 建模不同通信模式与网络配置受到约束。

为提供评估 DNN 系统的实用方案，我们提出 TrioSim。¹ TrioSim 接收在单块 GPU 上执行 PyTorch（可扩展到其他框架）时采集的算子级轨迹，并外推出更复杂的多 GPU 执行细节。

¹ <https://github.com/sarchlab/triosim>

TrioSim 的核心是轨迹外推器、算子级性能模型 [34] 与高层网络模拟器。轨迹外推器决定应由哪块 GPU 执行某一层的算子以及需要哪些数据。若数据不在该 GPU 上，轨迹外推器会添加通信任务，包括直接的内存搬移任务，以及与 NVIDIA 集合通信库（NVIDIA Collective Communications Library, NCCL）[50] 通信原语类似的集合通信任务。算子级性能模型方面，我们采用李氏模型（Li's Model）[34] 预测 DNN 算子与层的执行时间。最后，为估计数据搬移时间，我们构建了高层的基于流的网络模型：忽略协议细节，聚焦网络链路的延迟与带宽以及数据流之间的带宽共享（与基于流的网络模拟器 [12, 16] 类似）。

TrioSim 的新颖之处有两点。其一，TrioSim 只需单条 GPU 轨迹即可忠实模拟多 GPU 系统，能力超越现有模拟器。这种最小轨迹要求允许用户动态调整关键参数，包括批量大小、网络拓扑（如 NVSwitch [49]、mesh、胖树等）、并行策略（如数据并行 [33, 75]、张量并行 [65]、流水线并行 [25, 65]）与集合通信方案（如 NCCL 风格的归约 [50]），且无需额外采集轨迹。其二，这一能力源自 TrioSim 独特的轨迹外推方法，它在性能建模与模拟之间取得平衡。在高层，TrioSim 充当模拟器，基于系统状态重构未来事件；在底层，它集成李氏模型 [34] 做计算预测、集成基于流的网络模型做通信建模，确保执行时间估计的准确性。

本文的主要目标是交付一个极快的模拟器，能够建模海量多 GPU 平台上大规模 DNN 训练的性能（其局限性见第 8.4 小节）。本文的贡献包括：

1. **轨迹采集器（tracer）**。轨迹采集器在层或算子粒度收集 GPU 执行数据。我们特别强调其追踪张量信息、记录哪一层或哪个算子访问哪部分数据的能力。把计算轨迹与张量访问轨迹关联起来，为联合建模计算与数据搬移提供了关键信息。
2. **轨迹外推器**。轨迹外推器依据并行方案（如数据并行、张量并行、流水线并行）把单 GPU 轨迹转换为

多 GPU 轨迹。轨迹外推使无硬件条件下的多 GPU 执行模拟成为可能。

3. **模拟器 TrioSim**。我们把轨迹外推器、基于线性回归的性能模型与轻量级网络模型结合成 TrioSim。TrioSim 能以高性能与合理精度模拟多 GPU 系统上 DNN 的 GPU 执行，可在数秒内完成多个批次 DNN 训练的模拟。在 2 GPU 系统上预测 DNN 训练时间时，数据并行、张量并行与流水线并行的平均误差分别为 2.91%、4.54% 和 6.82%。

2 背景

2.1 多 GPU 平台上的 DNN 训练

多 GPU 平台被广泛用于加速 DNN 训练，因为多块 GPU 联合提供的计算吞吐量与内存容量高于单 GPU 平台 [46, 69]。要在 DNN 训练中充分利用多块 GPU，需要妥善放置数据、调度计算任务、跨 GPU 通信、同步进度并聚合结果的方案。下面介绍流行的并行 DNN 训练、GPU 间通信方案与性能建模方法。

并行 DNN 训练。并行 DNN 训练方案（概览见图 1）的核心是决定如何在 GPU 之间划分负载，策略因应对的挑战而异。数据并行（Data Parallelism, DP）把输入数据的批次分布到各 GPU，使每块 GPU 同时处理各自的批次。每块 GPU 各自对其数据批次做前向与反向传播，计算参数梯度；随后，GPU 间通信同步梯度以更新权重。张量并行（Tensor Parallelism, TP）把每个张量（如层的参数、层的输入）切分到多块 GPU 上并行处理，并在应用张量并行的层计算完毕后，各 GPU 相互通信收集全部设备上该层的部分输出，汇成完整结果。流水线并行（Pipeline Parallelism, PP）把 DNN 的各层分布到各 GPU，还可把每个小批量（mini-batch）进一步划分为若干微批量（micro-batch）。每块 GPU 处理分到的微批量，并把处理完的微批量传给下一块 GPU；当下一块 GPU 处理该微批量时，当前 GPU 开始处理下一个微批量，形成重叠的流水线并持续运行到全部微批量完成，实现连续的数据流动与跨 GPU 的并行执行。这些方法常组合为混合并行（Hybrid Parallelism, HP），以优化多 GPU 系统的可扩展性与资源利用率。

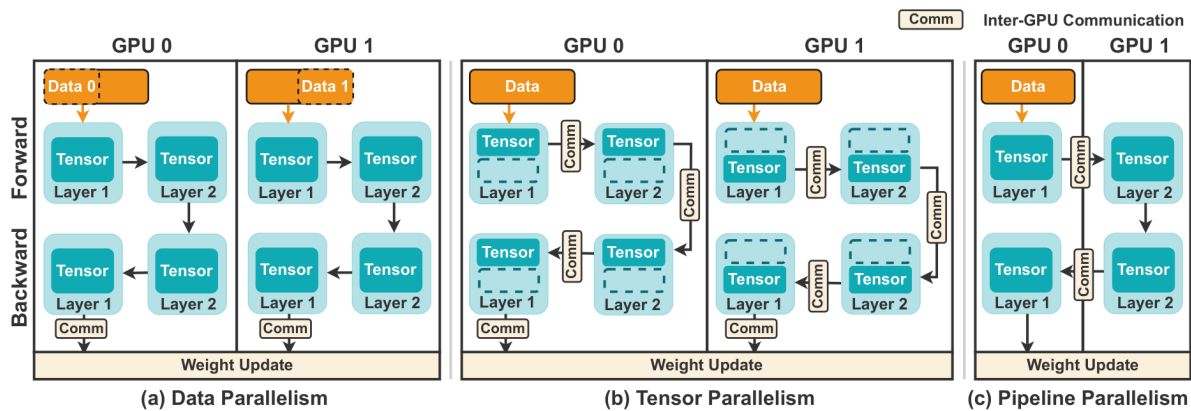


图 1 并行策略包括：(a) 数据并行，把数据集切分为多份并行处理，随后 GPU 通信同步以更新模型权重；(b) 张量并行，把张量切分为多份，每份交由一块 GPU 处理，之后 GPU 通信收集输出；(c) 流水线并行，把各层指派给各 GPU，每层计算后把输出发送给下一块 GPU。

图中文字说明：GPU 0/GPU 1 为两块 GPU；Tensor（张量）、Layer 1/Layer 2（层 1/层 2）、Forward（前向）、Backward（反向）、Comm（通信）、Weight Update（权重更新）、Inter-GPU Communication（GPU 间通信）、Data（数据）。

AllReduce 算法。AllReduce 算法是训练 DNN 时设备间通信的基础操作，在跨多块 GPU 或多个节点训练 DNN 时尤为普遍。AllReduce 旨在聚合全部参与处理器的数据并把结果分发回各处理器，确保每个处理器都拥有完整的结果集合。在 DNN 训练中平均梯度等场景下，这一操作至关重要——来自所有节点的数据同步与聚合是精确更新模型所必需的。AllReduce 的一种流行实现是基于环（ring）的算法，为分布式系统的数据聚合提供了高效且可扩展的方案。在环形 AllReduce 中，环拓扑上的每个节点依序向相邻节点发送并接收数据：每个节点把数据传给下一个节点，同时从环中的上一个节点接收数据；该过程持续到所有节点都拥有完整的聚合数据。环形方法把负载均匀分布到所有节点，最小化带宽占用。其通信模式可预测、实现简单，因此在各种高性能计算环境中被广泛采用。TrioSim 的轨迹外推器能生成环形 AllReduce 算法的任务序列，使 TrioSim 可以模拟集合通信。

多 GPU 体系结构。多 GPU 配置虽然带来了更高内存与计算容量的显著优势，但跨 GPU 的任务与通信协调仍是关键挑战。尤其地，GPU 间网络拓扑与互连技术的选择显著影响通信开销与整体性能。常见拓扑包括环、树、mesh 与总线。促成多 GPU 互连的现代实现包括 PCI Express（PCIe）[56]、NVLink [52] 与 NVSwitch [49]。PCIe 作为标准互连被众多系统广泛采用，通常组织为层次化的树状结构，以优化空间与时间上的数据流动。NVLink 提供了 PCIe 的高速替代，显著提升数据传输速率，并支持环与 mesh 等灵活拓扑，这对需要跨 GPU 快速共享数据的复杂计算任务至关重要。在 NVIDIA DGX-2 等系统中，NVLink 采用超立方 mesh 拓扑，将单机 8 块 GPU 连接起来，其中包含带

宽加倍的链路以强化连接并形成环路，为环形 AllReduce 算法提供最优支持。NVSwitch 进一步扩展 NVLink 的能力，实现任意到任意（any-to-any）的通信，通过全连接的网络拓扑消除传统互连中的瓶颈。

鉴于互连对系统性能的决定性作用，TrioSim 需要仔细建模网络行为。然而，由于我们的首要目标是高性能模拟，采用逐周期或逐微片（flit）模拟并不现实。因此，TrioSim 开发并集成了一个在细节与性能之间取得平衡的轻量级网络模型。

3 相关工作

GPU 性能分析与预测对优化 DNN 模型的效率至关重要。传统画像技术往往不足以刻画大规模场景下的 GPU 间交互 [2, 8, 19, 29, 51, 77, 78]，推动了多 GPU 系统上 DNN 性能建模方法的发展。我们分析潜在的解决方案，指出其优势与局限。

GPU 模拟器。评估 GPU 系统上 DNN 系统最直接的方法是使用周期级 GPU 模拟器（如 Accel-Sim [28]、Multi2Sim [72]、MGPUSim [67]、gem5 AMD GPU 模型 [62]）。然而，这些模拟器速度缓慢，不适用于大规模 DNN 负载与多 GPU 平台。模拟一个全尺寸 DNN 负载（如单 GPU 上的全尺寸 ResNet）可能需要数年 [1]。

基于采样的模拟。为加速 GPU 系统模拟，人们开发了基于采样的方法 [1, 40, 45]。这些方法基于代表性执行片段的详细模拟，选择性地跳过性能可以准确预测的部分模拟。例如，GPU 模拟器可以详细模拟内核中的部分线程束（warp），跳过执行相同指令集合的其余线程束 [40]。尽管有这些努力，基于采样的方法仍相对缓

慢且资源密集，主要因为非采样期间需要详细的全尺寸模拟。

GPU 缩尺模型。GPU 缩尺模型（scale-model）模拟 [64] 通过从小规模 GPU 性能数据外推来预测大规模 GPU 性能。缩尺模拟依赖预先画像的小规模模型（如含 4 个流式多处理器（Streaming Multiprocessor, SM）的 GPU），并假设可预测的扩展规律。然而，缩尺模型缺乏显式的通信建模，削弱了其对新体系结构与多 GPU 并行策略的适应性。相比之下，TrioSim 只需单条 GPU 轨迹，显式模拟 GPU 间通信，并支持可配置的网络拓扑与并行方案。缩尺模拟适用于高层 GPU 性能估计与体系结构级 GPU 研究，而 TrioSim 更适合系统级 GPU 研究（例如为复杂网络连接的 GPU 探索最佳并行策略）。

基于轨迹的 DNN 性能模型/模拟器。已有多个基于轨迹的性能模型用于评估多 GPU 系统上的 DNN 负载，包括 AstraSim [60, 74]、DistSim [41] 与 vTrain [3]。这些模型旨在利用执行轨迹而非详细的周期级模拟或繁重的多 GPU 画像来高效估计系统性能。我们在表 1 中详细比较这些工具与 TrioSim。

AstraSim [60, 74] 面向大规模深度学习训练，为分布式系统提供详细的通信建模。DistSim [41] 则专为混合并行探索而定制，支持多 GPU 执行建模。两款工具都很有价值，但它们需要多 GPU 或多节点画像轨迹，抬高了使用门槛。相比之下，TrioSim 只需单条 GPU 轨迹，显著减轻画像负担，让没有大规模硬件的用户也能使用。此外，TrioSim 采用模拟与性能建模的混合方法，提供更灵活的配置能力。例如，TrioSim 可以轻松支持非对称网络配置（如某条链路慢于其他链路），这在 AstraSim 与 DistSim 中建模和评估都很困难。

李氏模型 [34, 35] 提供内核级或算子级的解析模型，高效预测 GPU 执行时间。然而，它是逐内核或逐算子的模型而非完整模拟器，无法刻画系统级执行行为或 GPU 间通信动态。通过利用李氏模型，TrioSim 把其能力扩展到支持全尺寸 DNN 训练模拟，以更好地评估多 GPU 负载。

近期一个值得关注的工具是 vTrain [3]，它专为基于 Transformer 的大语言模型（Large Language

Model, LLM）设计，并利用 Transformer 层的重复性优化画像。然而，vTrain 有两个关键局限，使其无法完全满足社区的更广泛需求：(1) vTrain 专精于 LLM，虽然已在 512 GPU 系统上验证，但只用单一负载测试过，通用性受限；相比之下，TrioSim 提供更通用的 DNN 模拟平台，经多种负载验证，展示了更广的适用性。(2) vTrain 的轨迹要求严格：不仅需要多 GPU 硬件采集轨迹，还要求画像与模拟期间的批量大小和并行设置保持一致。这种僵硬的要求阻碍了完整的设计空间探索。相比之下，TrioSim 只需单条 GPU 轨迹，允许用户灵活配置网络拓扑、并行训练策略与批量大小，是更通用的平台。

特性	李氏模型 [34]	AstraSim [60, 74]	DistSim [41]	vTrain [3]	TrioSim (本工作)
目标负载	DNN 推理	DNN 训练	DNN 训练	Transformer 训练	DNN 训练
并行	不支持	DP、TP、PP	DP、TP、PP、HP	DP、TP、PP、HP	DP、TP、PP
网络	不支持	对称（如环、交换机）	基于画像	基于画像	灵活
轨迹要求	单 GPU	多 GPU	多节点	多节点	单 GPU
性能模型	解析式	以周期级模拟为主	解析式	解析式	解析式与模拟混合
支持新 GPU	是	否	否	否	借助李氏模型支持
宣称误差	7%（单 GPU） 15.2%（新 GPU）	N/A	<4%（多 GPU） <5%（单 GPU）	8.37%（单节点） 14.73%（多节点）	2.91%（DP）、 4.54%（TP） 6.82%（PP）

表 1 TrioSim 与类似性能建模工具在不同特性上的比较。

4 TrioSim

TrioSim 的设计（见图 2）包含以下主要功能：轨迹采集、多 GPU 轨迹外推、基于线性回归的时间建模与轻量级网络建模。TrioSim 用 Go 语言实现，构建于 Akita 模拟器引擎 [67] 之上。事件驱动的模拟引擎使我们能快速进无关细节、加速模拟。Akita 模拟器引擎还让 TrioSim 原生支持 AkitaRTM 的实时监控 [44] 与 Daisen 的可视化 [68]。

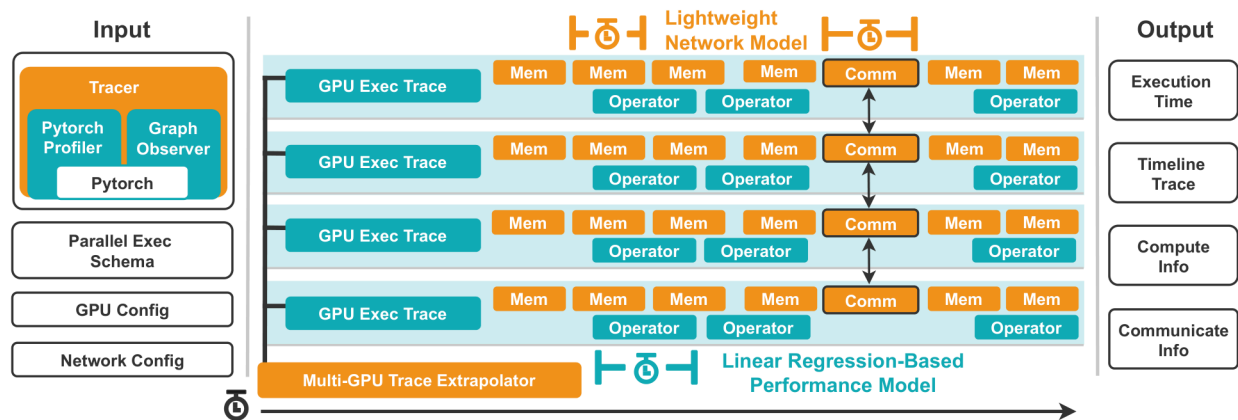


图 2 TrioSim 的设计包括轨迹采集、多 GPU 轨迹外推、基于线性回归的时间建模与轻量级网络建模。

图中文字说明：输入（Input）包括 PyTorch（PyTorch）、并行执行方案（Parallel Exec Schema）、GPU 配置（GPU Config）与网络配置（Network Config）；轨迹采集器（Tracer）由 PyTorch 剖析器（PyTorch Profiler）与执行图观察器（Graph Observer）构成，产生 GPU 执行轨迹（GPU Exec Trace）；多 GPU 轨迹外推器（Multi-GPU Trace Extrapolator）产出多份轨迹；基于线性回归的性能模型（Linear Regression-Based Performance Model）提供计算信息（Compute Info），轻量级网络模型（Lightweight Network Model）提供通信信息（Communicate Info）；输出（Output）包括执行时间（Execution Time）与时间线轨迹（Timeline Trace）；Mem、Operator、Comm 分别表示内存、算子与通信操作。

4.1 概述

TrioSim 的主要输入是轨迹采集工具生成的轨迹，其余输入包括网络拓扑、GPU 参数与并行方案。我们支持数据并行、张量并行与流水线并行等不同类型的并行。

获得输入配置后，多 GPU 轨迹外推器会在模拟过程中把单 GPU 轨迹分布到多个设备。基于线性回归的性能模型预测算子的计算时间，轻量级网络模型估计内存取回时间。在 GPU 之间同步时，通信过程由 TrioSim 发起，轻量级网络模型测量通信时间。

在输出方面，TrioSim 可返回不同并行下 DNN 在多块 GPU 上的总预测执行时间，也可返回负载中每层或每段的通信时间与计算时间。同时，它展示 GPU 间通信过程或每块 GPU 上计算过程的时间线。TrioSim 提供的其他信息包括每层输入输出张量的细节，以及 AllReduce 操作期间梯度计算所涉及的数据量。

4.2 轨迹采集器

轨迹格式。TrioSim 轨迹包含 DNN 训练过程中执行的 GPU 操作信息。我们要求每条记录包含算子名称、实测执行时间，以及以张量 ID 列表表示的输入/输出。此外，我们维护第二张表，记录 DNN 训练与推理过程用到的所有张量。除张量 ID 外，主要记录张量维度，以便在张量不在使用它的 GPU 上时估计需要搬移的字节数。

轨迹采集器设计。TrioSim 的轨迹采集器构建于最流行的 DNN 框架之一 PyTorch [55] 之上，主要依赖 PyTorch 剖析器（PyTorch Profiler）与执行图观察器（Execution Graph Observer）的输出来形成执行的完整视图。轨迹格式是标准化的，因此支持其他框架是可能的；我们把支持其他框架留作未来工作。

我们依靠 PyTorch 剖析器 [55] 提取 GPU 执行的算子及算子的执行时间。我们首先使用 PyTorch 剖析器记录的 CPU 轨迹，其中包含 DNN 各层与训练期间执行的基础算子（如矩阵乘法、矩阵转置）。PyTorch 剖析器还能记录所执行 GPU 内核及其开始与结束时间。我们用这些信息重建内核、算子与层之间的映射：把算子的开始时间视为其首个内核的开始时间，把结束时间视为其最后一个内核的结束时间。

此外，我们用执行图观察器 [38] 提取所执行 DNN 层的算子，以及 DNN 训练过程中相关的输入输出数据依赖。对每个算子，执行图观察器提供丰富的张量信息：1) 输入/输出张量列表；2) 张量类别（输入/权重/梯度/输出）；3) 张量格式（元素数据类型、维度）。

以层信息为桥梁，我们融合 PyTorch 剖析器与执行图观察器提供的信息，重建算子执行与内存访问两类信息。总体而言，TrioSim 的轨迹采集器捕获算子规格、算子执行时间与被访问张量的细节（见图 3）。

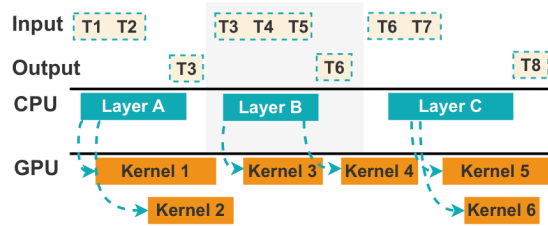


图 3 轨迹采集器中的轨迹采集。PyTorch 剖析器生成的轨迹显示 CPU 上的层与 GPU 上的内核，而执行图观察器提取 CPU 上的算子并识别输入输出的数据依赖。

图中文字说明：Layer A/B/C（层 A/B/C）、Kernel 1-6（内核 1-6）、CPU 与 GPU 两个时间轴、Input/Output（输入/输出）、T1-T8 为张量编号。

4.3 多 GPU 轨迹外推器

模拟多 GPU DNN 负载需要多 GPU 执行轨迹。然而，用户可能并不实际拥有想要评估的多 GPU 系统。因此，我们决定只要求用户提供单 GPU 执行采集的轨迹，并提供轨迹外推器，自动把单 GPU 轨迹转换为多 GPU 轨迹。整个外推过程随模拟展开而进行，额外的计算与通信算子在整个模拟过程中即时加入。

TrioSim 通过处理单 GPU 轨迹启动外推过程。直到第一个算子时，轨迹外推器决定哪些 GPU（可以是多块）需要执行该算子。假设需要 2 块 GPU 执行，TrioSim 随后检查这 2 块 GPU 的本地内存中是否有该算子所需数据；若无，TrioSim 就在复制出的计算算子之前插入数据搬移算子。插入完成后，TrioSim 处理插入的数据搬移与计算算子，把时间向前推进。虚拟时间（即模拟世界内的时间）的推进由计算性能模型（第 4.4 小节）与网络模型（第 4.5 小节）决定。处理完轨迹中的第一个元素后，TrioSim 转入下一个元素，直到轨迹中不再有元素。

用户可以指定并行执行与通信方案以及硬件连接拓扑。我们原生支持数据并行、张量并行与流水线并行训练。对于 GPU 间通信，我们支持 NCCL 风格的集合通信，包括归约（reduce）、散射（scatter）与收集（gather）过程。TrioSim 支持扩展网络拓扑、并行方案与集合通信方案，扩展灵活而直接。

数据并行训练。TrioSim 支持在多块 GPU 上模拟数据并行策略。对前向与反向传播过程，轨迹外推器只需复制全部计算算子。从前向传播收到梯度张量后，轨迹外推器为 AllReduce 操作添加必要的算子：既可与反向传播并行以节省执行时间，也可在反向传播之后执行。

TrioSim 允许使用与轨迹记录不同的批量大小，这对先前的模拟器（如 AstraSim、vTrain）并不容易。算子计算时间需要按比例缩放，由基于线性回归的性能模型（第 4.4 小节）估计；内存搬移时间也随之变化。该特性允许用户探索批量大小对训练速度与效率的影响。

张量并行训练。TrioSim 也支持在多块 GPU 上模拟张量并行策略。张量并行把权重矩阵切分到不同设备。每台设备计算其权重矩阵分片，然后从所有设备收集部分输出。我们对卷积、线性与嵌入等层模拟张量并行，因为 PyTorch 也是并行执行它们的。模拟以张量并行执行的层时，轨迹外推器把切分后的算子分布到每块 GPU 的队列，并在该层末尾追加必要的通信算子。TrioSim 随后按队列依次处理所有算子。

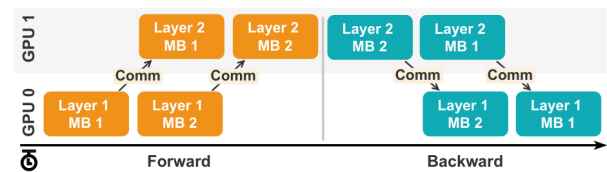


图 4 TrioSim 中的 GPipe。假设简单模型有两层，每块 GPU 分得一层，本例中 TrioSim 的 GPipe 实现把一个小批量划分为两个微批量。

图中文字说明：GPU 0/GPU 1 各承担 Layer 1/Layer 2（层 1/层 2）；MB 1/MB 2（微批量 1/微批量 2）；Comm（通信）；Forward（前向）、Backward（反向）为两个阶段。

流水线并行训练。在流水线并行的实现中，DNN 的不同层或段由 TrioSim 依据用户规格指派到不同设备。我们在 TrioSim 中实现了 GPipe（见图 4），因为它被主流框架广泛采用 [46, 57]。

基于 GPipe，轨迹外推器把小批量切分为等大的微批量，生成微批量计算算子。它最初把第一个微批量各层的计算算子指派给第一块 GPU；接着发起通信算子，让第一块 GPU 把输出发给下一块 GPU；同时，轨迹外推器在第一块 GPU 上开始下一微批量各层的计算算子。该循环不断重复，使微批量持续进入流水线。TrioSim 随后执行全部计算与通信算子，建立跨多块 GPU 的流水线并行并测量执行时间。

基于环的集合通信。TrioSim 原生支持面向环形网络的 NCCL 风格集合通信。在环拓扑中，每台设备连接左右两个相邻设备。TrioSim 把基于环的集合通信作为轨迹外推过程的一部分：当 GPU 之间通信时，把内存传输任务加入外推后的轨迹；这些任务随后由模拟器拾取，传输时间由网络模型（下文介绍）估计。

4.4 算子性能模型

TrioSim 逐个重放外推后的轨迹中的算子。要开始一个算子的执行，TrioSim 需要确保该算子要用的数据已在设备上；若没有，将先取回输入数据（需要网络模型估计时间）并分配输出数据。这里我们做两点假设：1) 一个张量总是存储在单一远端位置；2) 数据就绪之前执行不能开始。该假设对体系结构级研究未必成立（内核执行期间也可以加载数据），但对系统级研究普遍可以接受 [14, 17, 63]。一旦算子要访问的数据都在本地 GPU 上，算子执行即开始。TrioSim 不模拟算子的任何细节，只预测算子执行的耗时，直接快进到算子完成时刻。

TrioSim 提供两种预测算子执行时间的方案。较简单的一种直接使用轨迹提供的时间——即在单 GPU 平台上测得的执行时间。该方法准确但不灵活，因为我们无法改变批量大小（数据并行）或张量大小（张量并行）。当多 GPU 模拟的环境与参数和单 GPU 轨迹一致时，TrioSim 总是使用轨迹提供的算子执行时间。

更精细的一种方案是使用李氏模型 [34]，它无需大量计算即可展现很高的时间预测精度。TrioSim 把算子的输入/输出维度传给性能模型，让性能模型预测新算子（不同输入/输出大小）的执行时间。这样，TrioSim 通过比较浮点运算次数（FLOPs）的差异，就能用单 GPU 算子时间预测多 GPU 算子的时间，并把预测结果作为外推轨迹上新算子的执行时间。我们在主要使用李氏模型的同时，为支持 DNN 训练对它做了重要扩展。

4.5 网络建模

TrioSim 的目标是支撑大规模多 GPU 分布式 DNN 训练系统的设计。一个主要设计因素是连接 GPU 与计算节点的网络。因此，在 TrioSim 中开发或集成网络性能模型是必要的。

集成现有体系结构级网络模拟器 [26] 的一个主要顾虑是性能。计算机体系结构领域的大多数现有网络模拟器 [26] 以周期级细节建模网络，精度极佳但模拟缓慢。为了快速模拟网络，我们必须舍弃传输细节，聚焦影响性能的关键因素。在分组交换网络中，数据包被切分为微片（flit，一个周期内可发送的单元），微片彼此独立地传向目的地。链路一般可以视为流水线（级数与每级周期数共同决定吞吐量与延迟），便于建模与性能估计。

TrioSim 把一个数据包的传输视为 4 步过程：1) 路由、2) 带宽分配、3) 进度更新、4) 投递。当数据包开始发送时（见图 5），第一步是建立分组交换网络

的路由，我们用最短路径算法确定路由。路由确定后，为路由上的链路分配带宽。由于我们使用事件驱动的模拟引擎，可以在按 Little 定律 [39] 计算出的时刻调度一个潜在的投递事件，并假设此后带宽分配不再变化。事件调度使我们能跳过传输细节、快进模拟。若有新数据包开始或完成传输，我们就重新计算带宽，并重新调度当前所有在途数据包的投递事件。最后，在投递事件时刻，我们把数据包推入目的地缓冲区，通知接收方，并为其他数据包重新计算带宽分配。

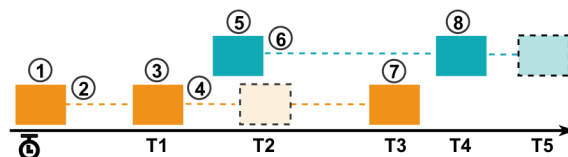


图 5 TrioSim 中分组交换网络的消息发送过程。情形 A：链路上只有一条消息。1) 新消息到来；2) 路由、带宽分配，并在时刻 T1 调度事件；3) 没有新消息到来，目的设备在时刻 T1 收到该消息。情形 B：链路上有两条消息。4) 把消息发往目的地，路由、带宽分配，并在时刻 T2 调度事件；5) 一条新消息开始传输，当前两条消息至少共享一条链路；6) 重新计算带宽，并在时刻 T3 与 T5 分别重新调度所有在途数据包的投递事件；7) 目的地在时刻 T3 收到橙色消息；由于橙色消息已被接收，重新计算带宽并在更早的时刻 T4 为蓝色消息重新调度投递事件；8) 目的地在时刻 T4 收到该消息。

图中文字说明：编号 1-8 对应图注所述步骤；T1-T5 为时间点标记。

虽然 TrioSim 默认支持分组交换网络，网络模型可以轻松替换。第 7.1 小节将展示 TrioSim 建模电路交换光子网络的能力。

5 实验方法

我们用一系列实验验证 TrioSim 的准确性，实验细节如下。

硬件环境。我们在三个平台（P1、P2、P3）上做验证。P1 有 2 块 NVIDIA A40 GPU，P2 有 4 块 NVIDIA A100 GPU，P3 有 8 块 NVIDIA H100 GPU。P1 的 GPU 经 PCIe 连接，P2 与 P3 的 GPU 经 NVLink 连接。GPU 与连接方式的多样性表明，TrioSim 能建模广泛配置下的性能。

用户可以在 TrioSim 中为链路设置任意带宽值。出于验证目的，我们需要提供与被验证硬件相匹配的合理带宽。我们发现链路的理论带宽通常派不上用场，因为实测带宽往往只是理论值的一小部分。为测量实测带宽，我们使用 nccltest [48]，并把测得的带宽作为模拟器中通信的输入参数。在全部实验中，我们对各平台使用同一组吞吐量数值。

软件环境。所有平台使用相同的系统配置：Python 与 CUDA 版本分别为 3.10.12 和 12.1。PyTorch 及相关库使用 torch 2.1.0+cu121、torchvision 0.16.0+cu121、torchaudio 2.1.0+cu121。对 Transformer 模型，由于 Transformer 训练的流水线并行仍在快速开发中，我们使用 nightly 版本：torch 2.5.0.dev20240623+cu121、torchvision 0.20.0.dev20240623+cu121、torchaudio 2.4.0.dev20240623+cu121。

负载。我们选择两类代表性 DNN。第一类是图像分类 DNN，包括 DenseNet [24]（DenseNet-121、DenseNet-161、DenseNet-169、DenseNet-201，图中记为 DN-121、DN-161、DN-169、DN-201）、ResNet [23]（ResNet-18、ResNet-34、ResNet-50、ResNet-101、ResNet-152，图中记为 RN-18、RN-34、RN-50、RN-101、RN-152）与 VGG [66]（VGG-11、VGG-13、VGG-16、VGG-19），全部来自 torchvision 库。第二类是主要用于自然语言处理（NLP）的 Transformer DNN，包括 GPT-2 [58]、BERT [9]（BERT-Base-Uncased）、T5 [59]（T5-Small）、FLAN-T5 [7]（FLAN-T5-Small）以及来自 Hugging Face [73] 的 Llama [13, 70, 71]（Llama-3.2-1B）。

并行策略。我们在真实硬件上用 PyTorch 库中标准数据并行、分布式数据并行（Distributed Data Parallelism, DDP）、张量并行与流水线并行训练 DNN 模型，并以此验证 TrioSim。

标准数据并行与分布式数据并行分别对应 PyTorch 提供的 DataParallel 与 DistributedDataParallel 模块。关键区别在于：分布式数据并行是多进程的，而标准数据并行是多线程的，这使分布式数据并行能避开 Python 解释器全局解释器锁（Global Interpreter Lock, GIL）造成的性能开销。与模拟器更相关的一个区别是：分布式数据并行让 NCCL 通信与层反向传播计算重叠，而标准数据并行要等全部反向传播结束才触发 NCCL 通信过程。

张量并行方面，我们使用开源库 BlackSamorez/tensor_parallel [5]。流水线并行方面，图像分类 DNN 使用 PyTorch 模块 torch.distributed.pipeline.sync.pipe；Transformer DNN 使用 torch.distributed.pipeline.sync.pipe。之所以对 Transformer 需要另一个模块，是因为 torch.distributed.pipeline.sync.pipe 要求把模型各层按顺序方式转换，以在并行之前定义期望的执行顺序，而它不支持实验集中的 Transformer 模型。两个

库都基于 GPipe 算法构建，与 TrioSim 中流水线并行的实现方法对应。

时间测量。我们用 PyTorch 提供的 torch.cuda.Event API 测量端到端执行时间：在每个批次执行前后打时间戳。每个批次运行 41 次迭代，前 30 个批次用作预热阶段。单 GPU 情形下，我们取预热后第 31 到 40 批的平均时间；用 PyTorch 剖析器收集第 41 批的层或内核时间信息。数据收集放在不同批次上进行，以防止画像开销影响所测性能。此外，我们用执行图观察器工具从第 41 批收集详细的输入、输出及其他张量或数据信息。对每个 DNN，我们还运行施加不同并行的多 GPU 情形，取 30 批预热之后第 31 到 40 批的平均执行时间，作为待验证的真值。注意，对使用张量并行的真实硬件时间，实验中我们只统计“至少一块 GPU 在忙或至少一个数据搬移任务在进行”的时间。我们不计 CPU 开销，因为 TrioSim 的主要目标是建模大规模 DNN 训练，此时 GPU 大概率被高度利用。

6 验证

我们给出一组实验结果来证明 TrioSim 的有效性，重点比较 TrioSim 预测的执行时间与真实硬件执行时间。

单 GPU 验证。首先验证 TrioSim 在单 GPU 训练上的准确性。我们用批量大小为 128 的单 GPU 轨迹，预测批量大小为 256 时的单 GPU 时间。图 6 的结果显示，所有模型的归一化时间（预测时间/真实硬件时间）都接近 1，表明 TrioSim 能对 A40 与 A100 都做出准确预测。全部受测模型的平均误差：A40 GPU 为 1.10%，A100 GPU 为 3.25%。

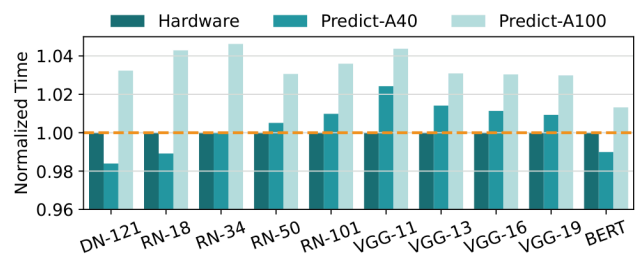


图 6 批量大小为 256、而只向 TrioSim 提供批量大小 128 采集的轨迹时，TrioSim 预测时间与真实硬件时间的比较。其他模型在真实硬件上批量大小 256 时会内存不足。A40 与 A100 的平均误差分别为 1.10% 与 3.25%。

图中文字说明：横轴为各模型（DN-121、RN-18、RN-34、RN-50、RN-101、VGG-11、VGG-13、VGG-16、VGG-19、BERT）；纵轴为归一化时间（Normalized Time）；图例 Hardware（真实硬件）、Predict-A40、Predict-A100 为预测结果。

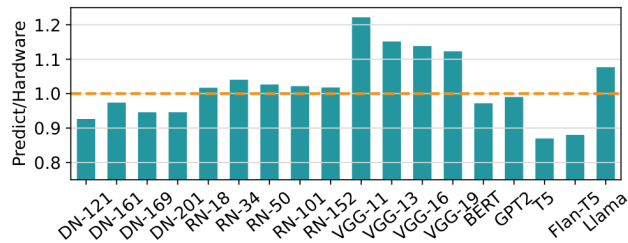


图 7 在 P1 上施加标准数据并行时，TrioSim 预测时间与真实硬件时间的比较。实验集的平均误差为 7.39%。

图中文字说明：横轴为各模型（DN-121 至 Llama）；纵轴为预测值/硬件值（Predict/Hardware）之比。

数据并行验证。其次验证 TrioSim 能否正确建模 PyTorch 在真实硬件上的标准数据并行。我们用 TrioSim 预测数据并行训练的性能，并与 P1 上真实硬件执行时间比较。结果（图 7）表明，TrioSim 能准确预测以标准数据并行训练的模型性能，实验集全部模型的平均误差为 7.39%。

接着，TrioSim 模拟真实硬件上 PyTorch `DistributedDataParallel` 的分布式数据并行。我们模拟 P1 与 P2 上分布式数据并行的训练性能，预测比标准数据并行更准确，误差（图 8）在 P1 与 P2 上分别低至 2.91% 与 2.73%。TrioSim 对分布式数据并行的预测更好——它被推荐用于替代标准数据并行，因为它让 NCCL AllReduce 与反向传播重叠。不过，标准与分布式数据并行的预测精度都足以满足学术研究需要。

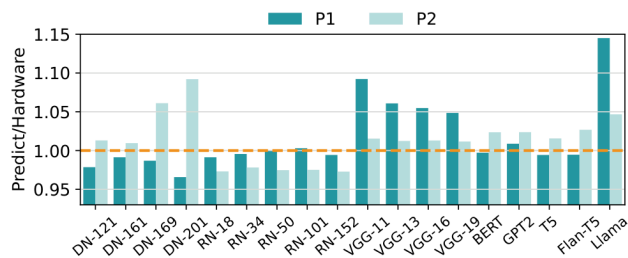


图 8 在 P1 与 P2 上施加分布式数据并行时，TrioSim 预测时间与真实硬件时间的比较。实验集在 P1 与 P2 上的平均误差分别为 2.91% 与 2.73%。

图中文字说明：横轴为各模型；纵轴为预测值/硬件值（Predict/Hardware）；图例 P1、P2 为两个平台。

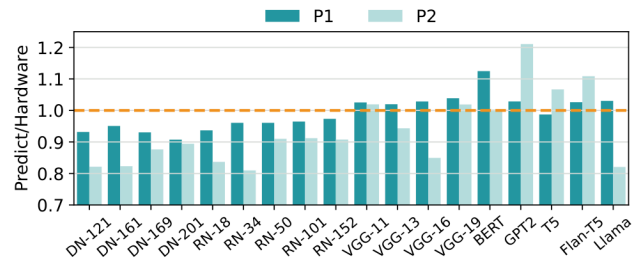


图 9 在 P1 与 P2 上施加张量并行时，TrioSim 预测时间与真实硬件时间的比较。实验集在 P1 与 P2 上的平均误差分别为 4.54% 与 11.24%。

图中文字说明：横轴为各模型；纵轴为预测值/硬件值（Predict/Hardware）；图例 P1、P2 为两个平台。

张量并行验证。TrioSim 也模拟采用张量并行的 DNN 训练。我们模拟 P1 与 P2 上张量并行的时间，平均误差（图 9）分别为 4.54% 与 11.24%。结果表明 TrioSim 能有效建模施加张量并行的负载性能。

流水线并行验证。我们在 P2 上分别用 2 块 A100 与 4 块 A100 模拟不同 DNN 的流水线并行。模拟期间，把每个小批量切分为微批量，微批量（块，chunk）数量取 1、2 和 4。微批量在不同流水线阶段并行处理。

我们向 TrioSim 输入批量大小 128 的单 GPU 轨迹（对 Llama，为避免真实硬件采集轨迹时内存不足，降至 16）。随着块数增加，GPU 上的微批量变小。微批量的减小会使 CPU 调度开销变得显著，导致端到端总执行时间变长，与“块越多执行时间应越短”的理论预期相反。例如对 DenseNet-169，P2 上 4 块的时间约为 2 块的 1.4 倍。因此这些情形不是 TrioSim 的主要目标，图 10 中柱顶带橙色三角标记的即属此类：2 块 A100 模拟中的 DenseNet-169 与 DenseNet-201，以及 4 块 A100 模拟中的 ResNet-18、ResNet-101、ResNet-152、DenseNet-169 与 DenseNet-201。

图 10 的结果表明，TrioSim 也能以合理精度处理流水线并行的模拟。2 块 A100 上，1 块、2 块与 4 块（chunk）模拟的平均误差分别为 6.82%、6.58% 与 15.10%；P2（4 块 A100）模拟的平均误差分别为 5.14%、8.96% 与 8.18%。

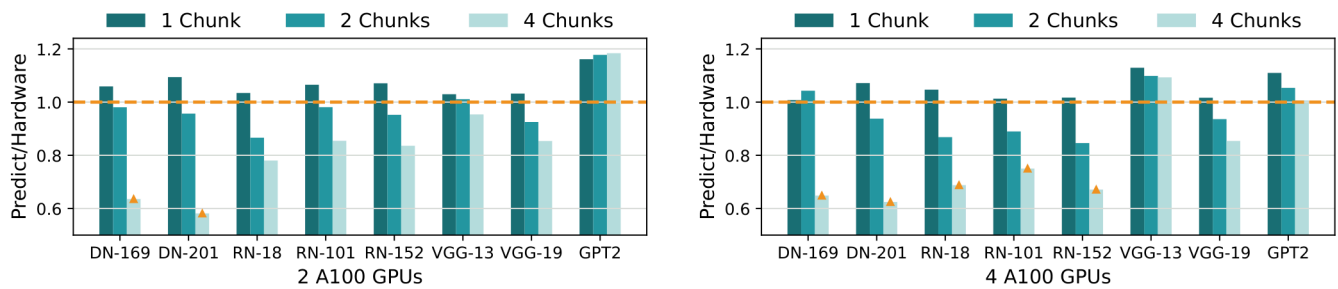


图 10 在 2 块 A100 与 4 块 A100 上以不同块数 (chunk) 施加流水线并行时, TrioSim 预测时间与真实硬件时间的比较。其他模型未展示, 因为需要扩展代码修改才能支持流水线并行。柱顶带橙色三角表示与“块越多执行时间应越短”的理论预期相反的异常值; 我们不处理此类情形, 因为它们不是 TrioSim 的主要目标。2 块 A100 实验集在 1、2、4 块时的平均误差分别为 6.82%、6.58% 与 15.10%; 4 块 A100 实验集在 1、2、4 块时的平均误差分别为 5.14%、8.96% 与 8.18%。

图中文字说明: 上下两组分别对应 2 A100 GPUs 与 4 A100 GPUs; 横轴为各模型; 纵轴为预测值/硬件值 (Predict/Hardware); 图例 1 Chunk、2 Chunks、4 Chunks 为块数。

Case 1: input (1x40 & 1xA100, BS128), output (8xH100, BS256); Case 2: input (1xH100, BS256), output (8xH100, BS256)

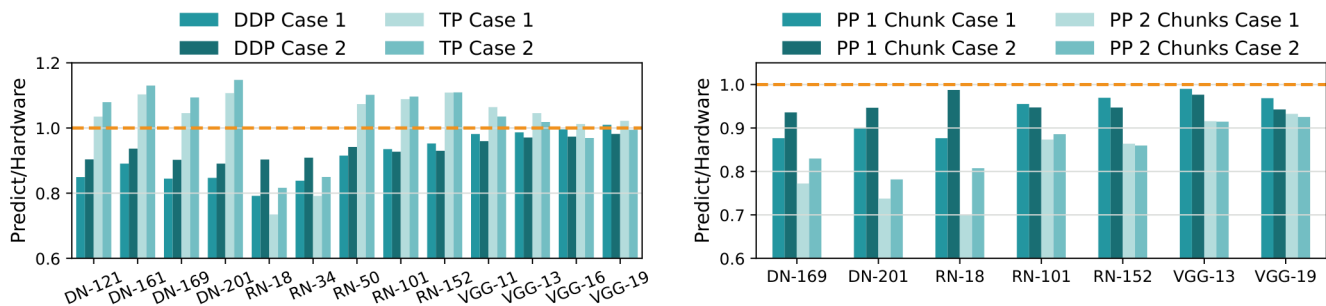


图 11 在 P3 上以不同块数施加分布式数据 (左)、张量 (左) 与流水线 (右) 并行时, TrioSim 预测时间与真实硬件时间的比较。每组四根柱: 两根为情形 1——用单块 A40 与单块 A100 采集的输入轨迹预测 8 H100 系统; 两根为情形 2——用单块 H100 采集的输入轨迹预测 8 H100 系统。

图中文字说明: Case 1 (情形 1) 输入为 1x40 与 1xA100、批量 128, 输出为 8xH100、批量 256; Case 2 (情形 2) 输入为 1xH100、批量 256, 输出为 8xH100、批量 256; DDP (分布式数据并行)、TP (张量并行)、PP (流水线并行) 各含 Case 1 与 Case 2 两种。

新 GPU 验证。我们在 P3 上验证多种并行策略, 展示 TrioSim 在 8 GPU 配置上的有效性。同时, 为表明 TrioSim 能预测新 GPU 与不同批量大小下的性能, 我们先用单块 A40 与 A100 GPU、批量 128 的轨迹——遵循李氏模型——预测 8 H100 系统在批量 256 下的性能; 作为对照, 我们也采集单块 H100 GPU 批量 256 的轨迹, 输入 TrioSim 预测 8 H100 系统的性能。

图 11 展示了 P3 上模型分别采用分布式数据并行、张量并行与流水线并行 (不同块数) 时, TrioSim 对真实硬件时间的预测误差。图中不含 Transformer 模型, 因为在真实硬件上采集轨迹会导致内存不足错误与不完整执行, 需要大量代码修改与调试。情形 1 (跨 GPU 预测) 中, 数据并行的平均误差为 9.09%, 张量并行为 9.07%, 流水线并行在 1 块与 2 块时分别为 5.65% 与 16.28%。情形 2 (同 GPU 预测) 中, 数据并行的平均误差为 6.69%, 张量并行为 9.09%, 流水线

并行在 1 块与 2 块时分别为 4.20% 与 13.76%。尽管跨 GPU 建模会引入误差, 误差仍保持相对较低。

并行策略比较。为证明 TrioSim 能在不同配置之间提供准确的相对比较, 我们在 P2 上实验比较各模型在数据、张量与流水线并行下的性能: 4 块 GPU 上固定总批量大小 128, 流水线微批量大小 64。结果 (图 12) 表明, TrioSim 总能预测各并行间的相对性能。真实硬件结果与 TrioSim 都支持: 当总负载恒定时, 数据并行是最高效的选择; 张量并行总体表现不佳, 但在 Transformer 上例外。此外, TrioSim 能对每个模型准确预测张量并行是否比流水线并行更高效。

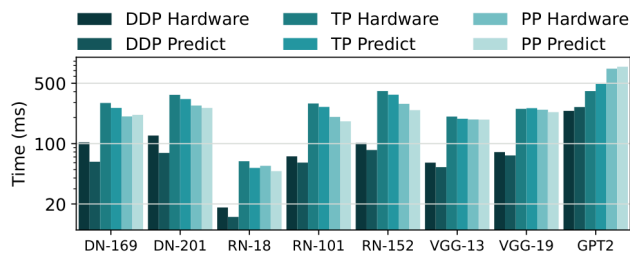


图 12 在 P2 上比较数据、张量与流水线并行，4 块 GPU 固定总批量大小 128，流水线微批量大小 64。

图中文字说明：横轴为各模型；纵轴为时间（Time，毫秒）；图例 DDP/TP/PP Hardware（真实硬件）与 Predict（预测）。

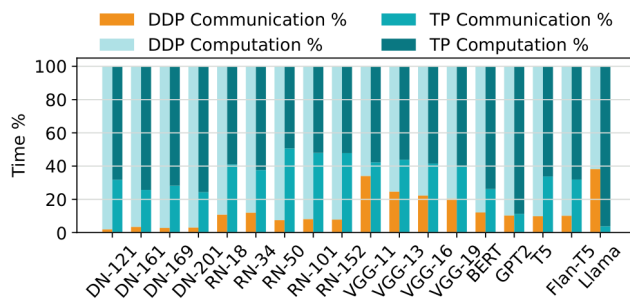


图 13 P1 上以张量并行与分布式数据并行模拟训练的模型，其通信与计算时间占比。

图中文字说明：横轴为各模型；纵轴为时间百分比（Time %）；图例 DDP Communication/Computation（分布式数据并行的通信/计算）与 TP Communication/Computation（张量并行的通信/计算）。

TrioSim 提供的通信与计算时间。TrioSim 能提供多 GPU 轨迹外推器的详细时间线轨迹。类似地，TrioSim 能提取并输出每个 DNN 模型的通信与计算时间（P1 上的通信与计算时间分解见图 13）。时间分解允许用户分析通信对总 GPU 时间的影响。结果表明，P1 上张量并行的通信时间占比高于数据并行。通过评估总计算与通信时间，用户可以就采用哪种并行策略、为多 GPU 系统确定最佳 GPU 数量做出有依据的决策。

TrioSim 执行时间。TrioSim 的模拟可在数秒内完成。图 14 展示了模拟器建模 P2 分布式数据并行时的执行时间。执行时间主要受系统中 GPU 总数与轨迹文件大小影响。

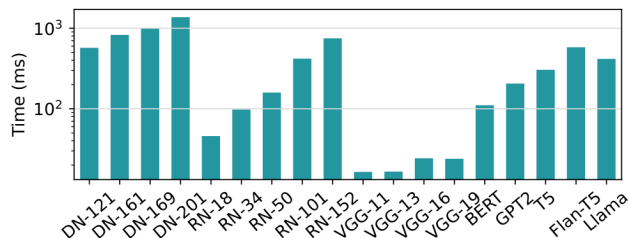


图 14 建模 P2 上分布式数据并行时，模拟器执行时间的对数刻度图。

图中文字说明：横轴为各模型；纵轴为时间（Time，毫秒， $10^2 - 10^3$ 刻度）。

7 案例研究

本节进行两个案例研究，展示 TrioSim 如何轻松评估新设计。

7.1 光子互连的晶圆级 GPU

此前的实验都用分组交换网络连接全部 GPU。然而，其他创新硬件网络可以提升传输效率。我们展示一个采用光子网络连接系统中 GPU 以降低通信开销的案例研究。我们为 TrioSim 开发光子网络模型来替代标准的分组交换网络模型。该案例研究展示了在 TrioSim 中建模用户自定义新网络的能力与低难度。

光子互连。光子互连 [15, 30, 36, 43] 以光作为数据传输介质。光信号的高频率与宽可用频带使这些光学链路能够提供高带宽与低延迟，同时保持节能的信息交换通道。

光子互连在多 GPU 系统上的一个著名应用是 Lightmatter 的 Passage [21]。Passage 提供一个光子中介层（interposer），连接晶圆上的多个芯粒（chiplet，如 GPU、加速器、内存池）。作为电路交换网络，Passage 需要建立占用底层光纤频带的逻辑链路。逻辑链路一旦建立，每条 Passage 连接即可提供高带宽与超低延迟。因此，无论物理距离远近，数据包几乎都能以相同延迟送达晶圆上的任何节点。

光子网络模型实现。TrioSim 只要求网络模型实现 Send 与 Deliver 两个函数，分别标记一次传输的开始与结束。Deliver 函数很直接，只需按引用把数据交给接收方；Send 函数则是 3 步过程：1) 建立链路、2) 预留缓冲区、3) 搬移数据。

发送数据时，我们先检查发送方与接收方之间是否已建立链路。若无链路可用，则花费一些时间（延迟可配置）通过调度模拟器事件来建立链路。由于 GPU 的光

子端口有限，若无可用端口，我们销毁闲置时间最长的未用链路。链路建立后，发送方等待目的地腾出缓冲区空间。注意该过程无需轮询，TrioSim 所用的 Akita 模拟器引擎支持在缓冲区空间可用时通知发送方。最后，一旦预留了缓冲区空间，我们就可以把投递时间计算为“数据大小除以带宽，加上链路延迟”，并按算得的传输时间调度投递事件。

把硬件平台配置为使用新互连非常简单：用户只需实例化光子互连类并调用 PlugIn 方法把设备端口与连接关联即可。由于数据传输独立于设备逻辑，无需修改设备代码。

硬件配置。由于 Passage 面向晶圆级系统，我们建模一个含 $12 \times 7 = 84$ 块 GPU 的晶圆，每块 GPU 等同一块 NVIDIA A100 GPU。该数值与 Cerebras 晶圆级引擎设计 [31] 的可曝光区 (reticle) 数量一致，且易于配置。实验中，我们把 Passage 配置为在 8 条链路上提供 484GB/s 带宽，并把建立链路的延迟设为 20 毫秒。对电气网络与光学网络，我们都用晶圆上的 mesh 网络构成环来执行 AllReduce 操作。

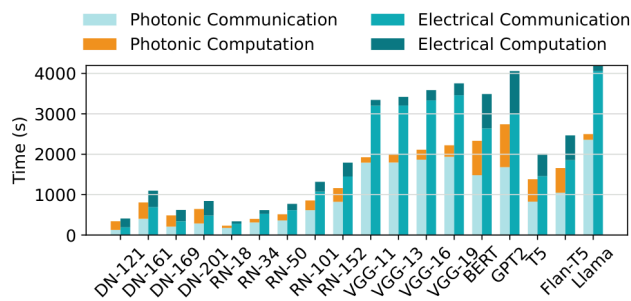


图 15 以数据并行训练 DNN 时， $12 \times 7 = 84$ 个芯粒经电气与光子互连连接的晶圆级 GPU 的性能。光学网络能显著减少通信时间。图中 Llama 的时间按 56 倍缩小显示。

图中文字说明：横轴为各模型；纵轴为时间（Time，秒）；图例 Photonic/Electrical Communication/Computation（光子/电气的通信/计算）。

结果。结果（图 15）表明，在这样的规模下，电气网络系统中通信显著影响执行时间。例如在 VGG-19 等网络中，通信占总执行时间的 92.21%，说明仅依赖数据并行训练对基于电气网络的晶圆级 GPU 并非最优。相比之下，光学网络把通信时间减少近一半。然而，该案例研究也表明，仅采用光子网络并不能完全解决可扩展性问题；要充分发挥光纤连接的晶圆级 GPU 的能力，还需要进一步设计通信方案。

7.2 用 Hop 进行异构训练

多个工作节点之间的异构性是分布式训练任务的关键挑战：在同步阶段，较快的工作节点必须等待较慢的节点，形成性能瓶颈。该问题对依赖 AllReduce 等同步通信方法的去中心化训练系统影响尤为深远，因为这些方法在设计时并未考虑异构性。TrioSim 的灵活性使其很适合研究异构多 GPU 系统。用其他类似工具 [3, 41, 60] 模拟这类系统颇具挑战，因为它们偏离所采集轨迹的能力有限。

本案例研究复现 Hop [42] 设计中的实验。Hop [42] 协议为去中心化训练引入了异构感知方法。Hop 的主要思想是使用基于队列的同步机制，智能化管理各工作节点之间的迭代时间差异。该机制通过更新队列（update queue）与令牌队列（token queue）实现：更新队列依据观察到的迭代差距动态调整大小，无需等待最慢节点即可高效协调；令牌队列则更严格地约束迭代差距，防止各工作节点状态过度发散。Hop 以若干策略特性增强去中心化训练，包括备份工作节点（backup worker）、有界陈旧度（bounded staleness）与跳过迭代。引入备份工作节点是为了在主工作节点成为瓶颈时保持产能；有界陈旧度利用基于队列同步的灵活性，允许更新中存在受控的陈旧度，在不损害收敛的前提下发挥陈旧同步并行模型的已知收益。

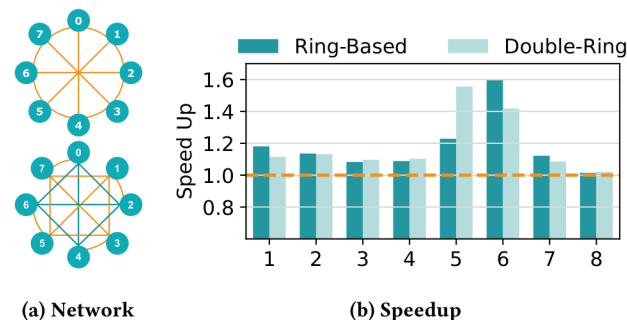


图 16 (a) 环形图（上）：GPU 以双向边连成环形拓扑，并在每个节点与其相距最远的节点之间额外建立一条连接。双环图（下）：两个环形图逐节点互连。(b) 横轴每组数据（共 8 组）代表 8 A100 系统中的一种随机减速场景；纵轴为异构多 GPU 系统中，环形与双环网络下引入一个备份工作节点所取得的加速比。图中文字说明：(a) Network（网络）中节点编号 0-7；(b) Speedup（加速比）；图例 Ring-Based（环形）、Double-Ring（双环）。

配置。我们在 TrioSim 中实现 Hop 协议，并进行了大量实验，评估备份工作节点对不同网络配置的影响。实验设置包括 8 块 A100 GPU，每次模拟 VGG-11 以批量大小 128 运行，与 Hop 论文中的负载规模一致。我们通过引入“把 GPU 通信带宽减慢 1 到 10 之间随机倍数”的机制来建模异构环境。实验在两类通信

图（见图 16：环形与双环）上进行。为探究备份工作节点的效果，我们在环形与双环配置中各引入一个备份工作节点，意味着每个节点可以错过一次更新而不影响整体进度。该配置展示了 TrioSim 模拟非标准网络与非对称 GPU 配置的能力。

结果。我们的数据集包含 8 组，代表 8 块 A100 GPU 上随机减速的场景。每组对应不同配置下的加速比：带一个备份工作节点与不带备份的环形网络，以及带一个备份工作节点与不带备份的双环网络。如图 16 所示，备份工作节点对性能的影响随减速比例的不同而大不相同。结果表明性能差异显著，证明备份工作节点如论文 [42] 所述，能在多样网络环境中增强去中心化训练系统的鲁棒性与效率。

我们在 TrioSim 中对 Hop 的实现展示了该模拟器的强大灵活性与适应不同网络状况和训练配置的能力。TrioSim 的体系结构允许用户集成各种同步策略，包括 Hop 提出的创新方法。这种适应性使其成为探索和验证 Hop 这类前沿分布式训练协议的理想平台。

8 讨论

8.1 TrioSim 是否足够精确

TrioSim 是为快速建模和预测大规模 DNN 训练执行时间而设计的工具，这类时间可能相差若干数量级。尽管如此，TrioSim 提供的估计非常有用，误差处于合理范围。TrioSim 的误差通常低于 20%，依负载与系统的复杂度，许多情形的误差不超过 10%。这样的误差率一般被认为足以支撑未来的学术研究。

8.2 误差来源

TrioSim 旨在为大规模 DNN 训练提供高效而准确的预测，但其精度会受某些误差来源与建模假设的限制。这些问题主要源于基于线性回归的性能模型、网络模型与多 GPU 轨迹外推器。

基于线性回归的性能模型。该模型依据并行策略（数据、张量或流水线并行）对批量大小进行缩放，通过外推单 GPU 轨迹来预测 GPU 执行时间。这一方法对 GPU 被充分利用的大规模负载效果良好，但在批量较小或模型轻量时会遇到困难。在这些条件下，GPU 可能对计算或内存资源施加的压力不足，导致预测性能不准。为解决该局限，TrioSim 允许集成其他计算模型，例如能更好处理利用不足负载的 NeuSight [32]，让用户能依据具体场景精化预测。

网络模型。网络模型用 Little 定律 [39] 模拟 GPU 间通信与内存搬移，基于带宽与延迟估计传输时间。然而，TrioSim 不建模通信协议，也不考虑数据传输单元大小不同或多条并发数据流带来的细微差别，这些可能引入轻微的不准确。这些局限使精确模拟细粒度网络行为变得困难。尽管如此，TrioSim 的高预测精度表明，它有效捕捉了延迟与带宽等最关键的因素，同时排除了影响较小的细节。

轨迹外推。多 GPU 轨迹外推器把单 GPU 轨迹扩展为多 GPU 执行的模拟，在为并行策略复制该过程时可能引入误差。例如在流水线并行中，模拟器自动把各层指派给 GPU 以平衡负载，这可能与真实系统的实际行为不一致。若层间相互依赖或通信瓶颈未被准确表达，就会出现偏差，导致模拟性能与实际性能出现偏差。

CPU 开销。TrioSim 只考虑 GPU 计算与数据搬移（包括 CPU 到 GPU 的数据搬移）。我们观察到，训练小负载时 CPU 可能成为性能瓶颈 [20]（如图 10 所示）。然而小负载不是 TrioSim 的主要目标；训练大负载时，数据搬移与层计算主导执行时间。估计 CPU 开销需要更复杂的 CPU 性能模型，有待专门的未来研究。

8.3 应如何使用 TrioSim

TrioSim 主要为系统级研究而设计，聚焦探索多 GPU 系统上大规模 DNN 负载的性能。TrioSim 不建模细粒度的体系结构变化，而是让研究者考察高层系统行为，例如并行策略、网络拓扑与扩展配置的影响。例如，给定一个 LLM 和特定的 GPU 互连拓扑，用户可以评估不同并行策略（数据、张量或流水线并行）以确定最高效的配置。由于 TrioSim 只用一条 GPU 轨迹就能准确建模多 GPU 执行，这种“单轨迹、参数可无限更改”的能力恰好契合该用途。

虽然 TrioSim 不支持基于数值的模拟或细粒度体系结构建模，但它可以与底层模拟器结合做体系结构级研究。如果某项硬件修改（如新的 TensorCore 设计或原子操作优化）影响特定 DNN 层，可行的做法是：先用周期精确模拟器（如 gem5 [62]、Accel-Sim [28] 或 MGPUSim [67]）模拟这些层，获得更新后的执行时间估计；再把这些调整后的执行时间馈入 TrioSim 的轨迹驱动外推框架，让用户观察体系结构变化在多 GPU 配置与端到端负载上的全规模影响。这种混合工作流实现了大规模性能评估——若仅用周期精确模拟器，因其高昂的计算成本将不可行。

8.4 局限性

TrioSim 的设计做了关键取舍，以有限的计算资源高效评估常规 DNN 负载。因此存在若干局限：(1) TrioSim 不支持功能仿真，因此无法建模依赖数值或高度不规则的负载。(2) 细粒度硬件修改无法被直接评估，因为 TrioSim 不是体系结构或周期级模拟器。(3) TrioSim 不采集与通信相关的内核轨迹，而是在外推过程中重建开源 NCCL 实现的行为。若未来 NCCL 发生变化，用户可能需要更新 TrioSim 以适应。(4) 受李氏模型 [34] 限制，TrioSim 假设 GPU 利用率较高，因此在 CPU 成为瓶颈或内核较小时精度较差。(5) 高度抽象的网络模型忽略了协议级效应与底层通信行为。(6) TrioSim 只在 CNN 与 Transformer 上做过验证。我们把支持其他类型负载（如强化学习、图神经网络）留作未来工作。尽管有这些局限，多样负载上观察到的高精度表明，TrioSim 能为设计大规模 DNN 系统提供可信的洞见与可靠的指引。

9 结论

本文介绍了 TrioSim，一个轨迹驱动的模拟器，能以高精度与短模拟时间模拟多 GPU 系统上的大规模 DNN 负载。在 LLM 极其流行的当下，快速建模此类大规模系统性能的能力正变得愈发关键。TrioSim 只需单 GPU 轨迹作为输入，显著降低了在该领域开展研究的硬件门槛。结合高层性能模型与网络模型，TrioSim 能模拟广泛 DNN、并行方式与 GPU 间网络的性能。TrioSim 体现了我们在详细模拟与底层模拟之间寻找中间道路、平衡模拟精度与性能的持续努力。

致谢

我们感谢匿名审稿人详细而富有建设性的反馈。本材料基于美国国家科学基金会（NSF）项目（#2402804、#2402805）支持的工作。部分工作在美国能源部托马斯·杰斐逊国家加速器装置的实验室定向研发计划下完成。

参考文献

- [1] Cesar Avalos Baddouh, Mahmoud Khairy, Roland N Green, Mathias Payer, and Timothy G Rogers. 2021. Principal Kernel Analysis: A Tractable Methodology to Simulate Scaled GPU Workloads. In *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*. 724–737.
- [2] Ammar Ahmad Awan, Hari Subramoni, and Dhableswar K Panda. 2017. An in-depth performance characterization of CPU-and GPU-based DNN training on modern architectures.

- In *Proceedings of the Machine Learning on HPC Environments*. 1–8.
- [3] Jehyeon Bang, Yujeong Choi, Myeongwoo Kim, Yongdeok Kim, and Minsoo Rhu. 2023. vtrain: A simulation framework for evaluating cost-effective and compute-optimal large language model training. arXiv preprint arXiv:2312.12391 (2023).
- [4] Yuhui Bao, Yifan Sun, Zlatan Feric, Michael Tian Shen, Micah Weston, José L Abellón, Trinayan Baruah, John Kim, Ajay Joshi, and David Kaeli. 2022. NaviSim: A Highly Accurate GPU Simulator for AMD RDNA GPUs. In *The 31st International Conference on Parallel Architectures and Compilation Techniques (PACT)*.
- [5] BlackSamorez. 2024. Tensor Parallel. Retrieved July 20, 2024 from https://github.com/BlackSamorez/tensor_parallel
- [6] Eric Chung, Jeremy Fowers, Kalin Ovtcharov, Michael Papamichael, Adrian Caulfield, Todd Massengill, Ming Liu, Daniel Lo, Shlomi Alkalay, Michael Haselman, et al. 2018. Serving dnns in real time at datacenter scale with project brainwave. *IEEE Micro* 38, 2 (2018), 8–20.
- [7] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, et al. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25, 70 (2024), 1–53.
- [8] Jinku Cui, Qidong Zhao, Yueming Hao, and Xu Liu. 2024. DrPy: Pinpointing Inefficient Memory Usage in Multi-Layer Python Applications. In *2024 IEEE/ACM International Symposium on Code Generation and Optimization (CGO)*. IEEE, 245–257.
- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding. arXiv preprint arXiv:1810.04805 (2018).
- [10] Aditya Dhakal, Sameer G Kulkarni, and KK Ramakrishnan. 2024. D-STACK: High Throughput DNN Inference by Effective Multiplexing and Spatio-Temporal Scheduling of GPUs. *IEEE Transactions on Cloud Computing* (2024).
- [11] Weiguang Ding, Ruoyan Wang, Fei Mao, and Graham Taylor. 2014. Theano-based large-scale visual recognition with multiple gpus. arXiv preprint arXiv:1412.2302 (2014).
- [12] Lukasz Drzewiecki and Monika Antoniak-Lewandowska. 2007. Flow Simulator—a flow-based network simulator. In *EUROCON 2007—The International Conference on “Computer as a Tool”*. IEEE, 2132–2136.
- [13] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. 2024. The llama 3 herd of models. arXiv preprint arXiv:2407.21783 (2024).
- [14] Yusuke Fujii, Takuya Azumi, Nobuhiko Nishio, Shinpei Kato, and Masato Eda. 2013. Data transfer matters for GPU computing. In *2013 International Conference on Parallel and Distributed Systems*. IEEE, 275–282.
- [15] Michael Georgas, Jonathan Leu, Benjamin Moss, Chen Sun, and Vladimir Stojanović. 2011. Addressing link-level

- design tradeoffs for integrated photonic interconnects. In *2011 IEEE Custom Integrated Circuits Conference (CICC)*. IEEE, 1–8.
- [16] Thomas J Giuli and Mary Baker. 2002. Narses: A scalable flow-based network simulator. arXiv preprint cs/0211024 (2002).
- [17] Chris Gregg and Kim Hazelwood. 2011. Where is the data? Why you cannot debate CPU vs. GPU performance without the answer. In *(IEEE ISPASS) IEEE International Symposium on Performance Analysis of Systems and Software*. IEEE, 134–144.
- [18] Udit Gupta, Carole-Jean Wu, Xiaodong Wang, Maxim Naumov, Brandon Reagen, David Brooks, Bradford Cottel, Kim Hazelwood, Mark Hempstead, Bill Jia, et al. 2020. The architectural implications of facebook’s dnn-based personalized recommendation. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 488–501.
- [19] Yueming Hao, Nikhil Jain, Rob Van der Wijngaart, Nirmal Saxena, Yuanbo Fan, and Xu Liu. 2023. DrGPU: A Top-Down Profiler for GPU Applications. In *Proceedings of the 2023 ACM/SPEC International Conference on Performance Engineering*. 43–53.
- [20] Yueming Hao, Xu Zhao, Bin Bao, David Berard, Will Constable, Adnan Aziz, and Xu Liu. 2023. TorchBench: Benchmarking PyTorch with High API Surface Coverage. arXiv:2304.14226 [cs.LG]
- [21] Nicholas C Harris, Darius Bunandar, Ajay Joshi, Ayon Basumallik, and Robert Turner. 2022. Passage: A wafer-scale programmable photonic communication substrate. In *2022 IEEE Hot Chips 34 Symposium (HCS)*. IEEE Computer Society, 1–26.
- [22] Kim Hazelwood, Sarah Bird, David Brooks, Soumith Chintala, Utku Diril, Dmytro Dzhulgakov, Mohamed Fawzy, Bill Jia, Yangqing Jia, Aditya Kalro, et al. 2018. Applied machine learning at facebook: A datacenter infrastructure perspective. In *2018 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 620–629.
- [23] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.
- [24] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4700–4708.
- [25] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems* 32 (2019).
- [26] Nan Jiang, George Micheliogiannakis, Daniel Becker, Brian Towles, and William J Dally. 2010. Booksim 2.0 user’s guide. *Stanford University* (2010), q1.
- [27] Norman P Jouppi, Cliff Young, Nishant Patil, David Patterson, Gaurav Agrawal, Raminder Bajwa, Sarah Bates, Suresh Bhatia, Nan Boden, Al Borchers, et al. 2017. In-datacenter performance analysis of a tensor processing unit. In *Proceedings of the 44th annual international symposium on computer architecture*. 1–12.
- [28] Mahmoud Khairy, Zhesheng Shen, Tor M Aamodt, and Timothy G Rogers. 2020. Accel-Sim: An extensible simulation framework for validated GPU modeling. In *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 473–486.
- [29] Heehoon Kim, Hyounghook Nam, Wookeun Jung, and Jaejin Lee. 2017. Performance analysis of CNN frameworks for GPUs. In *2017 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 55–64.
- [30] Ashok V Krishnamoorthy, Ron Ho, Xuezhe Zheng, Herb Schwetman, Jon Lexau, Pranay Koka, Guoliang Li, Ivan Shubin, and John E Cunningham. 2009. Computer systems based on silicon photonic interconnects. *Proc. IEEE* 97, 7 (2009), 1337–1361.
- [31] Mandy La and Andrew Chien. 2020. Cerebras Systems: Journey to the Wafer-Scale Engine. *University of Chicago, Tech. Rep* (2020).
- [32] Seonho Lee, Amar Phanishayee, and Divya Mahajan. 2024. Data-driven Forecasting of Deep Learning Performance on GPUs. arXiv preprint arXiv:2407.13853 (2024).
- [33] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. arXiv preprint arXiv:2006.15704 (2020).
- [34] Ying Li, Yifan Sun, and Adwait Jog. 2023. Path Forward Beyond Simulators: Fast and Accurate GPU Execution Time Prediction for DNN Workloads. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 380–394.
- [35] Ying Li, Yifan Sun, and Adwait Jog. 2023. A Regression-based Model for End-to-End Latency Prediction for DNN Execution on GPUs. In *the Proceedings of International Symposium on Performance Analysis of Systems and Software (ISPASS), Raleigh, NC*. 343–345.
- [36] Yuan Li, Ke Wang, Hao Zheng, Ahmed Louri, and Avinash Karanth. 2022. Ascend: A scalable and energy-efficient deep neural network accelerator with photonic interconnects. *IEEE Transactions on Circuits and Systems I: Regular Papers* 69, 7 (2022), 2730–2741.
- [37] Shao-Fu Lin, Yi-Jung Chen, Hsiang-Yun Cheng, and Chia-Lin Yang. 2023. Tensor Movement Orchestration in Multi-GPU Training Systems. In *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1140–1152.
- [38] Zhongyi Lin, Louis Feng, Ehsan K Ardestani, Jaewon Lee, John Lundell, Changkyu Kim, Arun Kejariwal, and John D Owens. 2022. Building a performance model for deep learning recommendation model training on gpus. In *2022 IEEE 29th International Conference on High Performance Computing, Data, and Analytics (HiPC)*. IEEE, 48–58.

- [39] John DC Little and Stephen C Graves. 2008. Little's law. *Building intuition: insights from basic operations management models and principles* (2008), 81–100.
- [40] Changxi Liu, Yifan Sun, and Trevor E Carlson. 2023. Photon: A fine-grained sampled simulation methodology for GPU workloads. In *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*. 1227–1241.
- [41] Guandong Lu, Runzhe Chen, Yakai Wang, Yangjie Zhou, Rui Zhang, Zheng Hu, Yanming Miao, Zhifang Cai, Li Li, Jingwen Leng, et al. 2023. DistSim: A performance model of large-scale hybrid distributed DNN training. In *Proceedings of the 20th ACM International Conference on Computing Frontiers*. 112–122.
- [42] Qinyi Luo, Jinkun Lin, Youwei Zhuo, and Xuehai Qian. 2019. Hop: Heterogeneity-aware decentralized training. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*. 893–907.
- [43] Christopher Monroe, Robert Raussendorf, Alex Ruthven, Kenneth R Brown, Peter Maunz, L-M Duan, and Jungsang Kim. 2014. Large-scale modular quantum-computer architecture with atomic memory and photonic interconnects. *Physical Review A* 89, 2 (2014), 022317.
- [44] Ali Mosallaei, Katherine Isaacs, and Yifan Sun. 2024. Looking into the Black Box: Monitoring Computer Architecture Simulations in Real-Time with AkitaRTM. In *The 57nd IEEE/ACM International Symposium on Microarchitecture*.
- [45] Mahmood Naderan-Tahan, Hossein SeyyedAghaei, and Lieven Eeckhout. 2023. Sieve: Stratified GPU-Compute Workload Sampling. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 224–234.
- [46] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 1–15.
- [47] Cristobal A Navarro, Nancy Hitschfeld-Kahler, and Luis Mateu. 2014. A survey on parallel computing and its applications in data-parallel problems using GPU architectures. *Communications in Computational Physics* 15, 2 (2014), 285–329.
- [48] NVIDIA. 2024. NCCL Tests. Retrieved July 20, 2024 from <https://github.com/NVIDIA/nccl-tests>
- [49] NVIDIA Corporation. 2018. NVSwitch Technical Overview. Retrieved July 25, 2024 from <https://images.nvidia.com/content/pdf/nvswitch-technical-overview.pdf>
- [50] NVIDIA Corporation. 2024. NVIDIA Collective Communications Library (NCCL). Retrieved November 15, 2024 from <https://developer.nvidia.com/nccl>
- [51] NVIDIA Corporation. 2024. NVIDIA Nsight Compute. Retrieved November 15, 2024 from <https://developer.nvidia.com/nsight-compute>
- [52] NVIDIA Corporation. 2024. NVIDIA NVLink. Retrieved July 18, 2024 from <https://www.nvidia.com/en-us/data-center/nvlink/>
- [53] John D Owens, Mike Houston, David Luebke, Simon Green, John E Stone, and James C Phillips. 2008. GPU computing. *Proc. IEEE* 96, 5 (2008), 879–899.
- [54] Saptadeep Pal, Eiman Ebrahimi, Arslan Zulfiqar, Yaosheng Fu, Victor Zhang, Szymon Migacz, David Nellans, and Puneet Gupta. 2019. Optimizing multi-GPU parallelization strategies for deep learning training. *Ieee Micro* 39, 5 (2019), 91–101.
- [55] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. 2019. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems* 32 (2019).
- [56] PCI-SIG. 2024. PCI Express Base Specification. Retrieved July 18, 2024 from <https://pcsig.com/specifications>
- [57] PyTorch Contributors. 2024. Distributed Pipelining. Retrieved July 27, 2024 from <https://pytorch.org/docs/main/distributed.pipelining.html>
- [58] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. 2019. Language models are unsupervised multitask learners. *OpenAI blog* 1, 8 (2019), 9.
- [59] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research* 21, 140 (2020), 1–67.
- [60] Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2020. Astra-sim: Enabling sw/hw co-design exploration for distributed dl training platforms. In *2020 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 81–92.
- [61] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. {Zero-offload}: Democratizing {billion-scale} model training. In *2021 USENIX Annual Technical Conference (USENIX ATC 21)*. 551–564.
- [62] Kyle Roarty and Matthew D Sinclair. 2020. Modeling modern gpu applications in gem5. In *gem5 Users Workshop*.
- [63] Christopher J Rossbach, Yuan Yu, Jon Currey, Jean-Philippe Martin, and Dennis Fetterly. 2013. Dandelion: a compiler and runtime for heterogeneous systems. In *Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles*. 49–68.
- [64] Hossein SeyyedAghaei, Mahmood Naderan-Tahan, and Lieven Eeckhout. 2024. GPU Scale-Model Simulation. In *2024 IEEE*

- International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 1125–1140.
- [65] Mohammad Shoneybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. arXiv preprint arXiv:1909.08053 (2019).
- [66] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. arXiv preprint arXiv:1409.1556 (2014).
- [67] Yifan Sun, Trinayan Baruah, Saiful A. Mojumder, Shi Dong, Xiang Gong, Shane Treadway, Yuhui Bao, Spencer Hance, Carter McCardwell, Vincent Zhao, et al. 2019. MGPU-Sim: Enabling Multi-GPU Performance Modeling and Optimization. In *46th International Symposium on Computer Architecture*.
- [68] Yifan Sun, Yixuan Zhang, Ali Mosallaei, Michael D Shah, Cody Dunne, and David Kaeli. 2021. Daisen: A Framework for Visualizing Detailed GPU Execution. In *Computer Graphics Forum*, Vol. 40. 239–250.
- [69] Alexey Svyatkovskiy, Julian Kates-Harbeck, and William Tang. 2017. Training distributed deep recurrent neural networks with mixed precision on GPU clusters. In *Proceedings of the Machine Learning on HPC Environments*. 1–8.
- [70] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. 2023. Llama: Open and efficient foundation language models. arXiv preprint arXiv:2302.13971 (2023).
- [71] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv preprint arXiv:2307.09288 (2023).
- [72] Rafael Ubal, Byunghyun Jang, Perhaad Mistry, Dana Schaa, and David Kaeli. 2012. Multi2Sim: A simulation framework for CPU-GPU computing. In *Proceedings of the 21st international conference on Parallel architectures and compilation techniques*. 335–344.
- [73] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. 2020. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*. 38–45.
- [74] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. Astra-sim2. 0: Modeling hierarchical networks and disaggregated systems for large-model training at scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 283–294.
- [75] Eric P Xing, Qirong Ho, Wei Dai, Jin-Kyu Kim, Jinliang Wei, Seunghak Lee, Xun Zheng, Pengtao Xie, Abhimanu Kumar, and Yaoliang Yu. 2015. Petuum: A new platform for distributed machine learning on big data. In *Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 1335–1344.
- [76] Yongkang Zhang, Haoxuan Yu, Chenxia Han, Cheng Wang, Baotong Lu, Yang Li, Xiaowen Chu, and Huaicheng Li. 2024. Missile: Fine-Grained, Hardware-Level GPU Resource Isolation for Multi-Tenant DNN Inference. arXiv preprint arXiv:2407.13996 (2024).
- [77] Keren Zhou, Yueming Hao, John Mellor-Crummey, Xiaozhu Meng, and Xu Liu. 2020. GVProf: A value profiler for GPU-based clusters. In *SC20: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 1–16.
- [78] Keren Zhou, Yueming Hao, John Mellor-Crummey, Xiaozhu Meng, and Xu Liu. 2022. ValueExpert: Exploring value patterns in GPU-Accelerated applications. In *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. 171–185.